

RoadSOS

Hackathon Submission Package

National Road Safety Hackathon 2026

CoERS — Centre of Excellence in Road Safety
IIT Madras

https://coers.iitm.ac.in/events/Hackathon/2026/rule_book/

Submission deadline: 31 May 2026 **Repository:**
<https://github.com/Arthrevs/RoadSOS>

Live app: <https://roadsos-frontend.vercel.app>

Section	Contents
1. Project overview	Problem, judging-criteria map
2. Assumptions	27 explicit design constraints
3. Software packages	Backend pip + frontend npm + external APIs
4. Architecture	Request flow, offline strategy, crash detection
5. Database documentation	RoadSOS — Structured Database Documentation
6. Key source code	Plus Code encoder + AI triage (full repo on GitHub)

Contents

1	Project overview	2
1.1	Judging-criteria map	2
2	Assumptions	2
2.1	Network and connectivity	2
2.2	Device and browser	2
2.3	Geolocation	3
2.4	Geography and data coverage	3
2.5	User behaviour and consent	3
2.6	Communications channels	3
2.7	AI triage	3
2.8	Hosting	4
2.9	Security and privacy	4
3	Software packages	4
3.1	Backend — Python runtime (backend/requirements.txt)	4
3.2	Backend — Python development tools (backend/requirements-dev.txt)	4
3.3	Frontend — npm runtime dependencies	4
3.4	Frontend — npm development dependencies	5
3.5	External services (consumed via REST — no SDK)	5
4	Architecture summary	5
4.1	Request flow — GET /search	5
4.2	Four-tier offline strategy	6
4.3	Crash detection	6
4.4	SOS dispatch	6
5	RoadSOS — Structured Database Documentation	6
6	RoadSOS — Structured Database Documentation	6
6.1	1. Emergency Numbers Database	7
6.1.1	1.1 Coverage	7
6.1.2	1.2 Schema	7
6.1.3	1.3 Example record	7
6.1.4	1.4 Data sourcing & verification	7
6.1.5	1.5 Update policy	8
6.2	2. OSM Category Classification Map	8
6.2.1	2.1 Schema	8
6.2.2	2.2 Full mapping	8
6.2.3	2.3 Category alignment with rulebook	9
6.3	3. Search Result Contact Object	9
6.3.1	3.1 Schema	9
6.4	4. Triage Request/Response	9
6.4.1	4.1 Request schema	9
6.4.2	4.2 Response schema	9
6.4.3	4.3 Priority matrix (rule-based fallback)	9
6.5	5. Dispatch Summary Request	10
6.5.1	5.1 Request schema	10
6.5.2	5.2 Response schema	10

6.6	6. Health Check Response	10
6.6.1	6.1 Schema	10
6.7	7. In-Memory TTL Cache	11
6.7.1	7.1 Instances	11
6.7.2	7.2 Cache key format	11
6.8	8. Frontend Local Cache (offlineDB.js)	11
6.8.1	8.1 Schema (localStorage value, JSON-encoded)	11
6.8.2	8.2 TTL	12
6.9	9. Service Worker Cache Strategy	12
6.10	10. Summary — All persistent structures	12
7	Key source code	12
7.1	Offline Plus Code encoder	13
7.1.1	frontend/src/utils/plusCodes.js [JavaScript, 3,825 chars]	13
7.2	AI triage logic	14
7.2.1	backend/services/ai_triage.py [Python, 7,582 chars]	14
7.3	Source index	18

1 Project overview

RoadSOS — emergency response and prioritised contact discovery for road accidents.

India records 1.5 lakh road-accident deaths per year. The first 60 minutes after a severe injury (the *golden hour*) determines survival. At a real crash scene a bystander currently needs 2–3 minutes just to find the right phone number via Google Maps. RoadSOS reduces that to under 10 seconds and works fully offline.

1.1 Judging-criteria map

Criterion (from rulebook)	How RoadSOS satisfies it
Number of emergency contacts found	OSM Overpass (9 categories, parallel) + Google Places (parallel, not sequential). Auto-expanding radius 5 km → 10 km → 20 km.
Reliability	Every external call wrapped in <code>_safe_*</code> helpers. 3-mirror Overpass with exponential back-off. API always returns HTTP 200 with a valid shape.
Offline functionality	Four-tier fallback: live backend → 24 h localStorage cache → bundled JSON (818 entries, 200 countries) → hardcoded mock. Country emergency numbers always offline.
International coverage	200 countries pre-loaded. ISO-3166 code from Nominatim. Emergency numbers switch automatically at borders.
Six mandatory service categories	<code>hospital</code> , <code>police</code> , <code>ambulance</code> , <code>towing</code> , <code>tyre</code> , <code>showroom</code> — all mapped to OSM tags and Google keyword queries.

2 Assumptions

The following assumptions underpin the design. The code degrades gracefully whenever any single assumption is violated at runtime.

2.1 Network and connectivity

- The user’s device has at least intermittent connectivity at first install so the service worker can precache the shell and bundled facilities. After install the app is fully usable offline.
- When online, OpenStreetMap Overpass and Google Places are reachable; when not, the four-tier fallback (live backend → localStorage cache → bundled JSON → hardcoded mock) ensures contacts are always returned.
- Render’s free-tier backend spins down after 15 min of inactivity; the first request takes up to 55 s. The frontend fires a warm-up ping and shows a status banner explaining this.

2.2 Device and browser

- Modern evergreen browsers (Chrome, Edge, Firefox, Safari 14). Service Workers, Geolocation API, and Web Speech API are required for full functionality.
- Battery Status API is supported on Chromium-based browsers but not iOS Safari or desktop Firefox. The battery row in the dispatch screen is hidden when the API is unavailable.

- Accelerometer access (crash detection) requires the DeviceMotion API. iOS Safari requires an explicit permission prompt; the app degrades gracefully when denied.

2.3 Geolocation

- GPS accuracy is within 50 m in open sky, 200 m in urban canyons. The app commits a coarse fix in under 2 s and refines with watchPosition.
- iCloud Private Relay and similar proxies can return IP-geolocated positions hundreds of kilometres from the device. A three-layer guard (org-name check, >500 km haversine drift, Apple datacentre bbox) rejects these and re-prompts for GPS.
- Locations are cached at ~110 m grid resolution (3 decimal places) coarse enough for high cache hit-rate, fine enough for emergency-contact relevance.

2.4 Geography and data coverage

- OSM data quality is best in India, Europe, and dense urban areas. In sparse rural regions the Overpass radius expands 8 km 25 km and falls back on bundled facilities (938 entries across 200 countries).
- Country emergency numbers are pre-bundled for all 200 countries and always render without a network call.
- ISO-3166 country code is resolved from Nominatim reverse-geocode; if Nominatim fails the app falls back to coarse bounding-box matching against the bundled country dataset.

2.5 User behaviour and consent

- Users grant geolocation permission on first launch; otherwise the app shows a Set-Location-Manually affordance.
- Medical ID data stored in localStorage is included in outgoing SOS payloads only after the user sets it up. It is never uploaded to RoadSOS servers.
- On auto-detected crash (sustained 40 km/h collapsing to 5 km/h within ~2.5 s, optionally confirmed by a 3.5 G accelerometer spike), a cancellation window lets the user abort false positives.

2.6 Communications channels

- Country code from Nominatim determines the primary dispatch channel: WhatsApp-dominant countries (India, Brazil, Indonesia, Nigeria, etc.) use wa.me links; elsewhere native SMS deep-links are used.
- If the WhatsApp app is not installed, the wa.me link opens WhatsApp Web. The handler deferred-falls-back to SMS after 1.2 s if the WA window failed to open.
- The window.open call for WhatsApp fires synchronously inside the click handler any await before it strips the user-gesture flag and causes browsers to silently block the popup.

2.7 AI triage

- Google Gemini 2.0 Flash is used for situational triage via direct REST (no SDK). Free-tier limits (60 RPM / 1500 RPD) are sufficient for a demo.
- If Gemini is unreachable or rate-limited the app falls back to deterministic rule-based prioritisation using the same two yes/no questions.

2.8 Hosting

- Frontend is on Vercel (auto-deploy from main). Backend is on Render (auto-deploy from main). Both are free tiers; production would migrate the backend to a paid host to eliminate cold starts.
- The frontend can be served from any static host; the backend is a vanilla FastAPI app deployable on any Python 3.11+ host with uvicorn.

2.9 Security and privacy

- Medical ID, language preference, and cached searches live in localStorage only. The backend is stateless and never persists user data.
- Rate limits (/search 30 rpm/IP, /triage 20 rpm/IP) are enforced by token-bucket middleware aware of Render's X-Forwarded-For header.
- Google Places API keys are server-side only; they are rotated per request; if a key returns 403 the rotation picks the next key. The frontend never sees them.

3 Software packages

3.1 Backend — Python runtime (backend/requirements.txt)

```
1 fastapi==0.115.0
2 uvicorn[standard]==0.30.6
3 httpx==0.27.2
4 python-dotenv==1.0.1
5 aiosqlite==0.20.0
6 pydantic==2.9.2
7 phonenumbers==8.13.46
8 unicode==1.3.0
```

3.2 Backend — Python development tools (backend/requirements-dev.txt)

```
1 -r requirements.txt
2 pytest==8.3.3
3 pytest-asyncio==0.24.0
4 # TestClient requires httpx (already in requirements) [U+2014] no extra dep needed
5 ruff==0.11.13
```

3.3 Frontend — npm runtime dependencies

Package	Version
i18next	^26.2.0
leaflet	^1.9.4
lucide-react	^1.16.0
react	^18.3.1
react-dom	^18.3.1
react-i18next	^17.0.8
react-joyride	^3.1.0
react-leaflet	^4.2.1
react-router-dom	^7.15.0
workbox-window	^7.4.1

3.4 Frontend — npm development dependencies

Package	Version
@types/react	^18.3.3
@types/react-dom	^18.3.0
@vitalets/google-translate-api	^9.2.1
@vitejs/plugin-react	^4.7.0
jsdom	^29.1.1
react-refresh	^0.18.0
vite	^7.3.3
vite-plugin-pwa	^1.3.0
vitest	^2.1.4
workbox-expiration	^7.4.1
workbox-precaching	^7.4.1
workbox-routing	^7.4.1
workbox-strategies	^7.4.1

3.5 External services (consumed via REST — no SDK)

Service / API	Role in RoadSOS
OpenStreetMap Overpass API	Public OSM data query. Three mirrors with exponential-backoff retry.
OpenStreetMap Nominatim	Reverse geocoding (lat/lon → country code, landmark).
Google Places API (Nearby Search + Place Details)	Unconditional parallel fallback (if configured).
Google Gemini 2.0 Flash	AI triage. Free tier: 60 RPM / 1 500 RPD.
CartoDB Dark Matter tiles	Map tile provider (no API key required).
Web platform APIs	Geolocation, DeviceMotion, Battery Status, Web Speech, Service Worker.

4 Architecture summary

The full 35-page technical reference is at [docs/TECHNICAL.pdf](#) in the repository. The summary below captures the request flow and the four-tier offline strategy.

4.1 Request flow — GET /search

1. Frontend issues GET /search?lat=...&lon=...&radius=5000.
2. Phase 1: parallel fan-out — Nominatim reverse geocode + Overpass query (9 OSM categories, 3 mirrors, 12s timeout each).
3. Phase 2: parallel Google Places Nearby Search if a key is configured (does **not** wait for OSM; rankby=distance for nearest-first).
4. Phase 3: merge + proximity dedupe (50m radius), then phone enrichment via up to 3 Place Details calls within a 5s budget.

5. Response: JSON with categorised contacts, country emergency numbers, landmark, ISO country code.
6. Frontend: render on Leaflet map (CartoDB Dark Matter tiles), populate Quick-Contacts dock, persist to 24 h localStorage cache.

4.2 Four-tier offline strategy

1. **Tier 1** — Live backend `/search` call (online path).
2. **Tier 2** — Service Worker + localStorage cache, 24 h TTL, keyed at ~ 1.1 km grid.
3. **Tier 3** — Bundled JSON shipped with the PWA: 818 entries across 200 countries.
4. **Tier 4** — Hardcoded mock so the UI never shows an empty state.

4.3 Crash detection

- **Signal 1 (primary)** — GPS velocity collapse: sustained ≥ 40 km/h $\rightarrow \leq 5$ km/h within 2.5 s. Tuned to reject ordinary braking and stop-and-go traffic.
- **Signal 2 (optional)** — Accelerometer spike: peak magnitude ≥ 3.5 G.
- **Confirmation:** the accelerometer spike, when motion permission is granted, must align within a 4 s window; a sustained-speed sudden stop alone still triggers.
- **Cooldown:** 12 s after confirmed event.
- **User override:** 10 s PIN-cancel window before auto-dispatch.

4.4 SOS dispatch

- WhatsApp-dominant countries (India, Brazil, Indonesia, Nigeria, ...): `wa.me` deep-link opened **synchronously** in the user-gesture frame.
- SMS-dominant countries: native `sms:` deep-link.
- Fallback: if the WA popup is closed within 1.2 s the handler fires an SMS fallback URL via `window.location.href`.
- After the synchronous dispatch: camera + torch + scene-photo capture, live tracking session creation, and DispatchScreen overlay all run asynchronously without blocking the open tab.

5 RoadSOS — Structured Database Documentation

The following is the structured database documentation required for the models.

6 RoadSOS — Structured Database Documentation

Submission artifact for Section 7.1 of the IIT Madras Road Safety Hackathon 2026 rulebook: > “Structured database used for the models are to be submitted.”

This document describes every persistent data structure used by RoadSOS, its schema, its source, and how it is consumed by the runtime.

6.1 1. Emergency Numbers Database

Authoritative source: backend/data/emergency_seed.json **Frontend bundle (generated):** frontend/src/utils/emergencyNumbers.js **Endpoint exposing it:** GET /offline-pack

6.1.1 1.1 Coverage

Metric	Value
Total countries & territories	200
UN-recognised states covered	100%
Asia	49
Europe	45
Africa	55
Americas	35
Oceania	16
File size (raw JSON)	~28 KB
File size (gzipped)	~9 KB

6.1.2 1.2 Schema

```
{
  "country":      "string    Human-readable English country name",
  "country_code": "string    ISO 3166-1 alpha-2 code, uppercase",
  "calling_code": "string    International dialling prefix incl. '+'",
  "police":       "string    National police emergency number",
  "ambulance":    "string    National ambulance / medical emergency number",
  "fire":         "string    National fire department emergency number",
  "general":      "string    Universal emergency number (often 112 / 911)"
}
```

6.1.3 1.3 Example record

```
{
  "country": "India",
  "country_code": "IN",
  "calling_code": "+91",
  "police": "100",
  "ambulance": "108",
  "fire": "101",
  "general": "112"
}
```

6.1.4 1.4 Data sourcing & verification

Primary source: **Wikipedia “List of emergency telephone numbers”** (canonical, government-maintained). Each entry was cross-checked against: 1. The country’s official government emergency services portal where available 2. ITU-T E.164 numbering plan documentation 3. EU-wide 112 standard for European jurisdictions

Where a country uses different numbers for different services, the most-publicised national number is recorded. For countries where 112 is universally accepted as a fallback (e.g. all GSM networks), **general** is set to 112.

6.1.5 1.5 Update policy

Static. Emergency numbers change roughly once per decade. Changes are made by editing `backend/data/emergency_seed.json` and regenerating the frontend bundle:

```
# Regenerate frontend/src/utils/emergencyNumbers.js from backend JSON
python scripts/sync_emergency_numbers.py
```

6.2 2. OSM Category Classification Map

Source: `backend/services/overpass_service.py` **CATEGORY_MAP** **Purpose:** Maps Open-StreetMap tag (key, value) pairs to the six visible RoadSOS contact categories.

6.2.1 2.1 Schema

```
CATEGORY_MAP: list[tuple[tuple[str, str], str]]
# [ ((osm_key, osm_value), roadsos_category), ... ]
```

6.2.2 2.2 Full mapping

OSM Key	OSM Value	RoadSOS Category	Notes
amenity	hospital	hospital	Tagged hospitals
amenity	clinic	hospital	Polyclinics, walk-in clinics
amenity	doctors	hospital	General practitioner offices
healthcare	hospital	hospital	Healthcare-namespaces hospitals
healthcare	clinic	hospital	Healthcare-namespaces clinics
amenity	police	police	Police stations
emergency	ambulance_station	ambulance	Dedicated ambulance bases
amenity	ambulance_station	ambulance	Alternate tagging
amenity	fire_station	ambulance	Fire stations (often run ambulances)
service:vehicle:recovery	yes	towing	Vehicle recovery / breakdown
service:vehicle:towing	yes	towing	Tow truck services
amenity	vehicle_recovery	towing	Dedicated recovery yards
shop	car_repair	repair	Mechanic / garage
amenity	car_repair	repair	Alternate tagging
shop	tyres	tyre	Tyre / puncture shops
shop	tyre	tyre	Alternate spelling
shop	car	showroom	Car dealerships / showrooms
shop	car_parts	showroom	Auto parts dealers

6.2.3 2.3 Category alignment with rulebook

The rulebook (Section 1.3.3 “Key Aspects for Coders to Include”) explicitly lists: - **Nearest Police Station** police category - **Hospitals** hospital category - **Ambulance services** ambulance category - **Towing services** towing category - **Nearest puncture shops** tyre category - **Showrooms** showroom category

All six rulebook-mandated categories are covered.

6.3 3. Search Result Contact Object

Source: backend/services/overpass_service.py **parse_element()** **Returned by:** GET /search, POST /triage, cached in localStorage

6.3.1 3.1 Schema

```
{
  "id":          "string    Stable unique ID, format 'osm_{id}' or 'gp_{place_id}'",
  "name":        "string    Establishment name (English preferred, falls back to local)",
  "category":    "enum      hospital | police | ambulance | towing | repair | tyre | showroom",
  "phone":       "string|null E.164-normalised phone, or basic-cleaned fallback",
  "distance":    "float      Great-circle distance from user in kilometres, 2-decimal",
  "lat":         "float      Latitude of establishment, WGS84",
  "lon":         "float      Longitude of establishment, WGS84",
  "source":      "string     'OpenStreetMap' or 'Google Places'",
  "isOpen":      "boolean|null Parsed from opening_hours; null if unparseable",
  "aiReason":    "string|null Set after /triage if AI prioritised this contact"
}
```

6.4 4. Triage Request/Response

Endpoint: POST /triage **Source:** backend/services/ai_triage.py

6.4.1 4.1 Request schema

```
{
  "injured":    "boolean    Are there injured persons on scene?",
  "blocking":   "boolean    Is the vehicle blocking traffic?",
  "contacts":   "Contact[]  Array of contact objects from /search"
}
```

6.4.2 4.2 Response schema

```
{
  "contacts":   "Contact[]  Same contacts, AI-reordered, top one has aiReason set",
  "reason":     "string      One-line explanation of the prioritisation"
}
```

6.4.3 4.3 Priority matrix (rule-based fallback)

injured	blocking	Top priority order
true	true	ambulance hospital police towing repair tyre showroom
true	false	ambulance hospital police towing repair tyre showroom
false	true	police towing repair ambulance hospital tyre showroom
false	false	repair tyre towing showroom police ambulance hospital

Within each tier, contacts are sorted by ascending distance.

6.5 5. Dispatch Summary Request

Endpoint: POST /dispatch-summary **Source:** backend/services/dispatch_service.py

6.5.1 5.1 Request schema

```
{
  "lat":      "float  Latitude, WGS84",
  "lon":      "float  Longitude, WGS84",
  "landmark": "string|null Reverse-geocoded landmark text",
  "injured":  "boolean Injured persons present",
  "blocking": "boolean Vehicle blocking traffic"
}
```

6.5.2 5.2 Response schema

```
{
  "summary": "string  Human-readable dispatch text, ready to read aloud or paste",
  "source":  "enum    'ai' (Gemini-generated) or 'template' (rule-based fallback)"
}
```

6.6 6. Health Check Response

Endpoint: GET /health **Source:** backend/services/health_service.py

6.6.1 6.1 Schema

```
{
  "status":      "string  'ok' when service is up",
  "uptime_seconds": "integer Seconds since process start",
  "configured": {
    "gemini":      "boolean  Gemini API key present",
```

```

    "google_places": "boolean  Google Places API key present"
  },
  "cache": {
    "overpass": { "size": "int", "hits": "int", "misses": "int", "hit_rate": "float" },
    "google": { "size": "int", "hits": "int", "misses": "int", "hit_rate": "float" },
    "geocode": { "size": "int", "hits": "int", "misses": "int", "hit_rate": "float" }
  }
}

```

6.7 7. In-Memory TTL Cache

Source: backend/services/cache.py **TTLCache Not persisted to disk** — rebuilt on each process start.

6.7.1 7.1 Instances

Cache	TTL	Max Entries	Cached payload
overpass_cache	3600 s (1 hour)	200	List of contact objects per (lat, lon, radius)
google_cache	3600 s (1 hour)	200	List of contact objects per (lat, lon, radius)
geocode_cache	86400 s (24 hours)	500	{landmark, country_code} per (lat, lon)

6.7.2 7.2 Cache key format

{lat:.4f}_{lon:.4f}_{suffix}

Coordinates are rounded to 4 decimal places (~11 m grid) so sub-meter GPS jitter does not cause cache misses.

6.8 8. Frontend Local Cache (offlineDB.js)

Source: frontend/src/utils/offlineDB.js **Storage:** localStorage **Purpose:** Last-known-good results when the user is fully offline.

6.8.1 8.1 Schema (localStorage value, JSON-encoded)

```

{
  "key": "string  '{lat:.2f}_{lon:.2f}' (1km grid)",
  "contacts": "Contact[]",
  "landmark": "string|null",
  "countryCode": "string|null",
  "timestamp": "integer  Unix milliseconds when cached"
}

```

6.8.2 8.2 TTL

24 hours. Older entries are returned with a “stale” indicator but not silently discarded — a stale cached contact is better than nothing in an emergency.

6.9 9. Service Worker Cache Strategy

Source: frontend/public/sw.js (Workbox) **Storage:** Cache Storage API (managed by browser)

Pattern	Strategy	Reason
/search*	NetworkFirst (5s timeout)	Fresh data preferred, cache is fallback
/triage*	NetworkFirst (5s timeout)	Same
/offline-pack	CacheFirst	Static seed; only updates with new deploys
/static/*	CacheFirst	App shell — versioned by Vite

6.10 10. Summary — All persistent structures

#	Artifact	Type	Size	Update cadence
1	Emergency numbers DB	Static JSON	200 entries	~once per decade
2	OSM category map	Source code	18 mappings	When OSM tags evolve
3	Contact object	Runtime schema	Per query	Live
4	Triage I/O	Runtime schema	Per query	Live
5	Dispatch I/O	Runtime schema	Per query	Live
6	Health response	Runtime schema	Per ping	Live
7	Server TTL cache	In-memory	Up to 900 entries	Auto-evicting
8	Browser localStorage	Per device	1 entry per area	7-day TTL
9	Service Worker cache	Per device	Up to 50 entries	NetworkFirst

RoadSOS does not store any user data on a server. No accounts, no telemetry, no user identifiers. All user state is local to the device.

7 Key source code

The two algorithmic centrepieces are included verbatim below: the fully-offline Plus Code (Open Location Code) encoder and the AI triage logic. The complete 146-file source tree and full Git history are on GitHub: <https://github.com/Arthrevs/RoadSOS>.

7.1 Offline Plus Code encoder

7.1.1 `frontend/src/utils/plusCodes.js` [JavaScript, 3,825 chars]

```

1  /**
2   * Open Location Code (Plus Codes) [U+2014] encode GPS coordinates to a short,
3   * speakable, dispatcher-friendly string. Algorithm by Google. Spec:
4   *   https://github.com/google/open-location-code/blob/main/docs/specification.md
5   *
6   * Why we ship this:
7   * - Indian emergency dispatchers (esp. 112 ERSS) accept Plus Codes.
8   * - "7M5237MC+37" is far easier to communicate by voice in a panicked
9   *   moment than "thirteen point zero eight two seven, eighty point two
10  *   seven zero seven".
11  * - **Fully offline** [U+2014] pure deterministic algorithm. No network, no
12  *   API key, no library.
13  *
14  * Implementation note:
15  * - Uses the reference INTEGER algorithm (multiply to a fixed precision,
16  *   then divide). Float-accumulation approaches drift on the final grid
17  *   digits, so we deliberately avoid them. Verified character-for-character
18  *   against Google's canonical encoder across 1000+ coordinates.
19  *
20  * Precision: length 10 (default) [U+2248] 14 m [U+00D7] 14 m square [U+2014] fine for a
21  *   ↪ crash site.
22  *
23  * Exports 'encodePlusCode(lat, lon)' [U+2192] 11-char code, e.g. "7M5237MC+37".
24  */
25  const ALPHABET = '23456789CFGHJMPQRVWX'; // 20 chars, skips 0/I/lookalikes
26  const SEPARATOR = '+';
27  const SEPARATOR_POSITION = 8;
28  const ENCODING_BASE = 20;
29  const LATITUDE_MAX = 90;
30  const LONGITUDE_MAX = 180;
31  const PAIR_CODE_LENGTH = 10;
32  const GRID_CODE_LENGTH = 5;
33  const GRID_COLUMNS = 4;
34  const GRID_ROWS = 5;
35
36  // Precision multipliers (integer algorithm).
37  const PAIR_PRECISION = ENCODING_BASE ** 3;
38  const FINAL_LAT_PRECISION = PAIR_PRECISION * GRID_ROWS ** GRID_CODE_LENGTH;
39  const FINAL_LON_PRECISION = PAIR_PRECISION * GRID_COLUMNS ** GRID_CODE_LENGTH;
40
41  function clipLatitude(lat) {
42    return Math.max(-90, Math.min(90, lat));
43  }
44
45  function normalizeLongitude(lon) {
46    let l = lon;
47    while (l < -180) l += 360;
48    while (l >= 180) l -= 360;
49    return l;
50  }
51
52  /**
53   * Encode (lat, lon) to a 10-digit Plus Code with the "+" separator.
54   *
55   * @param {number} lat [U+2014] degrees, -90 to 90 (clamped)
56   * @param {number} lon [U+2014] degrees, wrapped to [-180, 180)
57   * @returns {string} e.g. "7M5237MC+37", or "" for invalid input
58   */
59  export function encodePlusCode(lat, lon) {
60    if (typeof lat !== 'number' || typeof lon !== 'number'
61        || !isFinite(lat) || !isFinite(lon)) {
62      return '';
63    }
64
65    let latitude = clipLatitude(lat);
66    const longitude = normalizeLongitude(lon);
67
68    // Latitude 90 would overflow the top cell [U+2014] pull it inward by one cell.
69    if (latitude === 90) {
70      latitude = latitude - (ENCODING_BASE ** -3 / GRID_ROWS ** GRID_CODE_LENGTH);
71    }

```

```

72
73 // Convert to positive integers at the final precision. Integer-only math
74 // from here avoids floating-point drift on the trailing grid digits.
75 let latVal = Math.floor(
76   Math.round((latitude + LATITUDE_MAX) * FINAL_LAT_PRECISION * 1e6) / 1e6,
77 );
78 let lonVal = Math.floor(
79   Math.round((longitude + LONGITUDE_MAX) * FINAL_LON_PRECISION * 1e6) / 1e6,
80 );
81
82 // A length-10 code uses only the pair section, so drop the grid digits.
83 latVal = Math.floor(latVal / GRID_ROWS ** GRID_CODE_LENGTH);
84 lonVal = Math.floor(lonVal / GRID_COLUMNS ** GRID_CODE_LENGTH);
85
86 // Build the 5 lat/lon pairs from least- to most-significant.
87 let code = '';
88 for (let i = 0; i < PAIR_CODE_LENGTH / 2; i++) {
89   code = ALPHABET.charAt(lonVal % ENCODING_BASE) + code;
90   code = ALPHABET.charAt(latVal % ENCODING_BASE) + code;
91   latVal = Math.floor(latVal / ENCODING_BASE);
92   lonVal = Math.floor(lonVal / ENCODING_BASE);
93 }
94
95 // Insert the "+" separator after position 8.
96 return code.slice(0, SEPARATOR_POSITION) + SEPARATOR + code.slice(SEPARATOR_POSITION);
97 }
98
99 /**
100  * Build a Google Maps shareable URL for given coordinates.
101  * Used by SOS-by-SMS so the recipient can tap-to-open the location.
102  */
103 export function gMapsLink(lat, lon) {
104   return `https://maps.google.com/?q=${lat.toFixed(6)},${lon.toFixed(6)}`;
105 }

```

7.2 AI triage logic

7.2.1 backend/services/ai_triage.py [Python, 7,582 chars]

```

1  """AI-driven contact prioritisation.
2
3  Uses Google's Gemini 2.5 Flash for situation-aware reordering. Always falls
4  back to a deterministic rule-based ordering if the API is unreachable, errors,
5  or returns malformed output. The fallback produces the same response shape so
6  the frontend never sees a difference.
7
8  Why fallback matters: emergency software cannot have its core feature dependent
9  on a 3rd-party API.
10
11  Why Gemini Flash: free tier (15 RPM, 1500 RPD on 2.5-flash, 15 RPM on 1.5-flash),
12  low latency, JSON-mode supported, no billing required to start.
13  """
14
15  from __future__ import annotations
16
17  import json
18  import logging
19  import os
20  import re
21
22  import httpx
23
24  from services.gemini_quota import gemini_quota
25  from services.gemini_utils import extract_gemini_text
26
27  logger = logging.getLogger(__name__)
28
29  # Gemini 2.5 Flash [U+2014] current free-tier default with the highest free quota.
30  MODEL = "gemini-2.5-flash"
31  MAX_TOKENS = 2048
32  TIMEOUT_S = 15.0
33
34  # REST endpoint (no SDK dependency [U+2014] keeps requirements lean and matches our
35  # existing httpx-everywhere style).

```

```

36 GEMINI_URL = (
37     "https://generativelanguage.googleapis.com/v1beta/models/{model}:generateContent?key={
38         ↪ key}"
39 )
40 SYSTEM_PROMPT = """You are RoadSOS Triage [U+2014] an emergency dispatcher AI for road
41     ↪ accidents.
42
43 Your only job: take a crash situation and a list of nearby emergency services,
44 return a JSON object that reorders the services by priority for that specific
45 situation. Uses Gemini 2.5 Flash to reorder the contacts for the specific situation.
46
47 OUTPUT REQUIREMENTS (strict):
48 - Return ONLY a JSON object. No prose. No markdown fences. No commentary.
49 - The object MUST have exactly two keys: "contacts" and "reason".
50 - "contacts" MUST be the same array of objects you were given, with all fields
51   preserved verbatim, just reordered. Never invent contacts. Never drop them.
52 - "reason" MUST be a single sentence, maximum 18 words, plain English,
53   explaining why the top contact was chosen for this situation.
54
55 PRIORITY RULES (in strict order):
56 1. Injured + vehicle blocking road [U+2192] ambulance [U+2192] hospital [U+2192] police [U
57   ↪ +2192] towing [U+2192] repair
58 2. Injured + no road block [U+2192] ambulance [U+2192] hospital [U+2192] police [U
59   ↪ +2192] repair
60 3. Not injured + blocking road [U+2192] police [U+2192] towing [U+2192] repair [U
61   ↪ +2192] tyre
62 4. Not injured + no block [U+2192] repair [U+2192] tyre [U+2192] police [U+2192]
63   ↪ towing
64
65 Within a priority tier, the closer service wins."""
66
67 _FENCE_RE = re.compile(r"~'~'(?:(?:json)?\s*|\s*'~'$", re.MULTILINE)
68
69 def rule_based_triage(injured: bool, blocking: bool, contacts: list[dict]) -> dict:
70     """Deterministic ordering used as fallback and as a baseline for tests."""
71     if injured and blocking:
72         order = ["ambulance", "hospital", "police", "towing", "repair", "tyre", "showroom"]
73         reason = "Trauma care plus blocked road [U+00B7] ambulance, hospital, police listed
74             ↪ first"
75     elif injured:
76         order = ["ambulance", "hospital", "police", "repair", "towing", "tyre", "showroom"]
77         reason = "Trauma care prioritised [U+00B7] ambulance and hospital listed first"
78     elif blocking:
79         order = ["police", "towing", "repair", "tyre", "hospital", "ambulance", "showroom"]
80         reason = "Vehicle blocking traffic [U+00B7] police and towing listed first"
81     else:
82         order = ["repair", "tyre", "showroom", "police", "towing", "hospital", "ambulance"]
83         reason = "No injuries reported [U+00B7] roadside repair services listed first"
84
85     def priority(c: dict) -> tuple:
86         cat = c.get("category", "")
87         idx = order.index(cat) if cat in order else 99
88         return (idx, c.get("distance", 9999))
89
90     return {
91         "contacts": sorted(contacts, key=priority),
92         "reason": reason,
93     }
94
95 def _validate_ai_result(result: object, original_count: int) -> dict | None:
96     """Return the result if it matches the contract, else None."""
97     if not isinstance(result, dict):
98         return None
99     if "contacts" not in result or "reason" not in result:
100         return None
101     if not isinstance(result["contacts"], list):
102         return None
103     if not isinstance(result["reason"], str) or not result["reason"].strip():
104         return None
105     if len(result["contacts"]) != original_count:
106         return None
107     return result

```

```

106
107 async def prioritize_contacts(injured: bool, blocking: bool, contacts: list[dict]) -> dict:
108     if not contacts:
109         return {"contacts": [], "reason": "No nearby services found"}
110
111     api_key = os.getenv("GEMINI_API_KEY", "")
112     if not api_key:
113         logger.info("GEMINI_API_KEY not set [U+2014] using rule-based triage")
114         return rule_based_triage(injured, blocking, contacts)
115
116     if not gemini_quota.can_call():
117         logger.info("Gemini quota guard triggered [U+2014] using rule-based triage")
118         return rule_based_triage(injured, blocking, contacts)
119
120     try:
121         situation = []
122         if injured:
123             situation.append("PEOPLE ARE INJURED and need medical attention")
124         if blocking:
125             situation.append("THE VEHICLE IS BLOCKING THE ROAD")
126         if not situation:
127             situation.append("no injuries, vehicle may need roadside assistance")
128         situation_text = "; ".join(situation)
129
130         summary = [
131             {
132                 "name": c.get("name", "?"),
133                 "category": c.get("category", "?"),
134                 "distance_km": c.get("distance", "?"),
135             }
136             for c in contacts
137         ]
138
139         user_message = (
140             f"SITUATION: {situation_text}.\n\n"
141             f"Services found nearby (currently sorted by distance only):\n"
142             f"{json.dumps(summary, indent=2)}\n\n"
143             f"Full contact data to reorder (return ALL of these, "
144             f"just in the priority order you decide):\n"
145             f"{json.dumps(contacts, indent=2)}"
146         )
147
148         # Gemini puts the system prompt in 'systemInstruction', not in messages.
149         # JSON mode is opt-in via responseMimeType.
150         payload = {
151             "systemInstruction": {"parts": [{"text": SYSTEM_PROMPT}]},
152             "contents": [
153                 {"role": "user", "parts": [{"text": user_message}]},
154             ],
155             "generationConfig": {
156                 "temperature": 0.2,
157                 "maxOutputTokens": MAX_TOKENS,
158                 "responseMimeType": "application/json",
159                 "thinkingConfig": {"thinkingBudget": 0},
160             },
161         }
162
163         url = GEMINI_URL.format(model=MODEL, key=api_key)
164         async with httpx.AsyncClient(timeout=TIMEOUT_S) as client:
165             r = await client.post(url, json=payload)
166             r.raise_for_status()
167             response_json = r.json()
168             await gemini_quota.record_call()
169
170         raw = extract_gemini_text(response_json)
171         if not raw:
172             logger.warning("AI returned empty response [U+00B7] using rule-based fallback")
173             return rule_based_triage(injured, blocking, contacts)
174
175         cleaned = _FENCE_RE.sub("", raw).strip()
176
177         try:
178             parsed = json.loads(cleaned)
179         except json.JSONDecodeError:
180             logger.warning("AI returned non-JSON [U+00B7] using rule-based fallback")
181             return rule_based_triage(injured, blocking, contacts)
182

```

```
183     validated = _validate_ai_result(parsed, len(contacts))
184     if validated is None:
185         logger.warning("AI returned malformed response [U+00B7] using rule-based
↳ fallback")
186         return rule_based_triage(injured, blocking, contacts)
187
188     return validated
189
190 except Exception as exc:
191     logger.warning(f"AI triage call failed ({type(exc).__name__}) [U+00B7] using rule-
↳ based fallback")
192     return rule_based_triage(injured, blocking, contacts)
```

7.3 Source index

Files included verbatim: **2**.

Document version. Generated from `main`. Re-run `python scripts/build_submission_tex.py` after any code change to regenerate.